Anderson Contreira
Posted on Dec 4, 2025

Edit Manage Stats

RESTful – Pílula 6 – Paginação, Filtros e Ordenação em APIs REST

#braziliandevs #rest

Conforme uma API cresce, lidar com grandes volumes de dados se torna inevitável. Para manter performance, organização e consistência, APIs RESTful maduras utilizam padrões bem definidos de **paginação**, **filtros**, **seleção de campos (fields)** e **ordenação**.

Esses mecanismos tornam a API mais eficiente, previsível e flexível para integrações externas.

Por que isso é importante?

- evita respostas gigantes e lentas
- melhora performance do servidor e do banco
- reduz tráfego desnecessário
- aumenta a flexibilidade para consultas complexas
- oferece controle total para o consumidor da API

Paginação – Os padrões mais utilizados

A API pode adotar mais de um padrão, mas deve manter consistência.

---

1. Modelo baseado em page / limit

Requisição:

```
GET /users?page=1&limit=20
```

Resposta recomendada:

```
{
"success": true,
  "code": 1,
  "label": "OK",
  "message": "Operation completed successfully",
  "params": ["page", "limit"],
  "data": [
    { "id": 1, "name": "Alice" },
    { "id": 2, "name": "Bob" }
  ],
  "meta": {
    "page": 1,
    "limit": 20,
    "total": 143,
    "totalPages": 8
  }
}
```

Esse é o modelo mais intuitivo e comum.

---

2. Modelo baseado em offset / limit

Requisição:

```
GET /users?offset=0&limit=20
```

Resposta recomendada:

```
{
  "success": true,
  "code": 1,
  "label": "OK",
  "message": "Operation completed successfully",
  "params": ["offset", "limit"],
  "data": [
    { "id": 1, "name": "Alice" },
    { "id": 2, "name": "Bob" }
  ],
  "meta": {
    "offset": 0,
    "limit": 20,
    "total": 143,
    "count": 2
  }
}
```

Características:

- `offset` indica o deslocamento inicial
- `limit` indica quantos registros retornar
- `count` mostra quantos itens vieram nesta resposta

Muito utilizado em bancos SQL e sistemas que precisam navegar por índices exatos.

---

3. Cursor-based pagination

Ideal para bases muito grandes ou altamente dinâmicas.

Exemplo:

```
GET /orders?cursor=eyJpZCI6MTB9&limit=20
```

Benefícios:

- evita inconsistências entre páginas
- escala melhor
- recomendado quando registros mudam com frequência

Filtros — Consultas mais flexíveis

Filtros permitem buscar apenas dados relevantes, sem criar endpoints específicos.

Exemplos:

```
GET /users?status=active&role=admin
```

Filtros mais elaborados:

```
GET /products?category=clothes&min_price=50&max_price=200
```

Evite endpoints com lógica embutida:

```
GET /users/active        // não recomendado
GET /products/cheap      // não recomendado
```

Seleção de campos (fields)

Permite ao cliente escolher **quais campos** deseja receber, reduzindo tamanho do payload e melhorando performance.

Requisição:

```
GET /users?fields=id,name,email
```

Resposta:

```
{
"success": true,
  "code": 1,
  "label": "OK",
  "message": "Operation completed successfully",
  "params": [
"fields"
],
  "data": [
    { "id": 1, "name": "Alice", "email": "alice@example.com" },
    { "id": 2, "name": "Bob",   "email": "bob@example.com" }
  ],
```

```
  "meta": {
    "offset": 0,
    "limit": 20,
    "total": 143,
    "count": 2
  }
}
```

Validação do fields

A API deve:

- impedir retorno de campos inexistentes
- bloquear campos privados ou sensíveis
- padronizar erros quando parâmetros inválidos forem enviados

---

Erro recomendado para campos inválidos

A API deve retornar erro claro, consistente e seguro quando o cliente pedir um campo inexistente ou proibido.

```
{
  "success": false,
  "code": 32,
  "label": "INVALID_FIELD_ERROR",
  "message": "Invalid field in 'fields' parameter",
  "params": [
    "fields"
  ],
  "details": "Allowed fields are: id, name, email, created_at",
  "trace": "Traceback (most recent call last):\n  File \"/usr/local/lib/pytho
}
```

Notas importantes sobre segurança e padronização

Sobre o código interno (code = 32)

- Ele complementa o status HTTP (neste caso, 400).
- Toda API madura mantém uma tabela de códigos internos padronizados.
- Facilita debugging, monitoramento, suporte e internacionalização.

Sobre o label

A `label` é a chave mais importante para o frontend:

- é estável
- é semântica
- permite mapeamento para diferentes idiomas
- possibilita UX customizada em cada aplicativo

Exemplo de mapeamento frontend:

```
{
  INVALID_FIELD_ERROR: {
    en: "Invalid field.",
    pt: "Campo inválido.",
    es: "Campo inválido."
  }
}
```

Sobre o campo trace

- nunca deve ser exposto em produção
- útil apenas em logs internos ou dev mode
- stack traces expostos aumentam superfície de ataque

Sobre o campo details

- deve ajudar a entender o erro
- sem expor lógica interna, queries, tabelas, caminhos de arquivo
- mensagens simples como “Allowed fields are…” são aceitáveis

Ordenação (sorting)

A API pode adotar dois padrões amplamente aceitos.

---

1. Padrão com prefixo "-"

```
GET /users?sort=created_at
GET /users?sort=-created_at
```

- `field` → ordem crescente
- `-field` → ordem decrescente

Simples e muito comum.

---

2. Padrão sort + order_by (mais explícito)

```
GET /users?sort=name&order_by=ASC
GET /users?sort=name&order_by=DESC
```

Vantagens:

- mais claro
- amigável ao consumidor
- fácil de validar

Exemplos:

```
GET /products?sort=price&order_by=ASC
GET /products?sort=created_at&order_by=DESC
```

Ambos os padrões são válidos; escolha um e mantenha consistência.

Combinando tudo em uma única consulta

```
GET /orders?status=paid&sort=created_at&order_by=DESC&offset=20&limit=20&fiel
```

Boas práticas gerais

- mantenha consistência em toda a API
- documente filtros, campos e ordenação suportados
- evite endpoints com lógica embutida
- sempre valide campos em `fields`
- nunca exponha stack traces em ambiente produtivo
- considere cursor pagination para bases muito grandes

Resumo

- Paginação evita sobrecarga e melhora performance
- Page/limit e offset/limit são válidos
- Cursor pagination é excelente para grandes volumes
- Filtros trazem flexibilidade
- Seleção de campos ajuda na performance e organização
- Ordenação pode ser prefixada (`-created_at`) ou explícita (`order_by=ASC`)
- Erros devem ser padronizados com `code`, `label`, `message`, `params`, `details` e controle seguro de `trace`

Top comments (2)

Lucas Lamounier • Dec 4 '25

Que artigo excelente, Anderson! A abordagem passo a passo sobre paginação, filtros e ordenação ficou muito clara. Adorei a parte sobre cursor-based pagination - muito relevante para APIs que precisam escalar. Parabéns pelo detalhe! 👋

Anderson Contreira • Dec 4 '25

Obrigado pelo comentário. Ah, sim, tem muita coisa legal, tem detalhes que fazem toda a diferença, por isso estou compartilhando um pouco desse assunto de que eu gosto bastante, eu mesmo não cheguei a usá-lo, mas certamente é uma solução muito interessante em cenários de alta demanda, ajudando a manter a consistência das respostas/páginas.

Code of Conduct • Report abuse

Anderson Contreira

Solutions Architect. IT professional since 2008. Over 6 years of leadership experience. Specialist in distributed architecture, microservices, cloud (AWS/GCP), DevOps and Full-Stack experience.

LOCATION
Brazil

EDUCATION
Distributed Software Architecture @ PUC Minas

WORK
Cloud Software Archtect @ Caylent

JOINED
Oct 31, 2025

More from Anderson Contreira

ALERTA IMPORTANTE PARA DEVS: golpe em testes técnicos está crescendo - fiquem espertos
#braziliandevs #scam #recruiting #development

Planejamento de carreira: ou você assume o volante, ou vira passageiro da própria vida
#braziliandevs #career

Por que o plano de carreira virou exceção, e como isso afeta a entrada de novos profissionais no mercado

#braziliandevs #career